

AUDIT BEFORE ACT

A deterministic pre-execution audit for the reasoning behind an onchain action. Live on Base mainnet.

TWO GATES. ONE IS MISSING.

GATE 1 · the transaction

Is this transfer safe? Is the contract real?
Is the slippage sane?

Well covered. Wallets, simulators,
Blockaid, smart-transaction pipelines.

GATE 2 · the thesis

Is there a sourced probability?
A falsification condition?
Is the edge real, or consensus in costume?

Uncovered. Reasoning is not a transaction
and does not look dangerous until it is.

An agent can pass every transaction check and still be wrong about why it is trading.

A PRICED, DETERMINISTIC VERDICT

Submit a trade thesis in free text. Get back a verdict, the defects that produced it, and a reproducible hash.

- **PASS / WAIT / FAIL**
- Fired taxonomy codes (DJZS-M: narrative gap, falsification absent, probability unsourced, consensus-as-edge)
- A verdict_hash anyone can recompute from the thesis text
- A ProofOfLogic certificate anchored to permanent storage

2.00 USDC

per audit, on Base

Paid over x402.

Out-of-scope requests
are refused free.

The engine is deterministic: the model emits boolean flags, TypeScript decides the score. Same thesis, same verdict.

// STATUS

LIVE ON BASE MAINNET

MCP transport	mcp.djzs.ai/mcp	live, paid
HTTP x402 transport	mcp.djzs.ai/x402/verify	live, paid
Onchain contracts	TrustScore, EscrowLock, Staking, AgentRegistry	deployed
Certificates	ProofOfLogic anchored to Irys	anchoring

Any HTTP x402 client pays the same gate: Base MCP, MetaMask Agent Wallet, x402-axios.

// EVIDENCE

FREE REFUSAL, PROVEN ON CHAIN

A refused audit must cost nothing. That is not a policy statement here. It is a zero delta.

out-of-scope intent submitted	engine returns in_scope: false
payer USDC before	10.185017
payer USDC after	10.185017
delta	0.000000

Verified against Base mainnet, not against the client library reporting success.

Paid audits on record settle 2.00 USDC to treasury and write an onchain trust score for the calling agent.

// DISTRIBUTION

REFERENCE INTEGRATION, PUBLISHED

djzs-polymarket-gate audits a Polymarket CLOB trade thesis and creates the order only on a clean in-scope PASS. WAIT, FAIL, and out-of-scope all halt before an order object exists.

capital action -> GATE 1 DJZS audits the THESIS -> PASS -> GATE 2 wallet audits the TRANSACTION -> onchain

|
WAIT / FAIL / out-of-scope -> HALT, no order created

github.com/SIFR0-dev/djzs-polymarket-gate

Open source. The gate is a dependency, not a rewrite: it sits at the seam between thesis formation and order placement.

WHAT IS PROVEN. WHAT IS NOT.

PROVEN

- Both transports live and payable on mainnet
- Paid audits settled, verified on chain
- Free refusal proven by balance delta
- Reference integration published

NOT YET

- Paid calls from external wallets: zero
- The gate is payable, not yet paid by others
- Polymarket paid execution path unexercised
- Distribution is the open problem, not the build

A protocol that audits reasoning cannot overstate its own. This slide is the product demonstrating itself.

THE AGENT STACK IS ON BASE

- x402 is the payment rail agents already speak, and it settles on Base. Every paid audit is a USDC settlement here.
- The wallet surfaces that need an upstream reasoning gate (Agent Wallet, Base MCP) are Base-native.
- Trust scores and ProofOfLogic certificates are written to Base, so an agent record is portable across venues.
- Every agent transacting on Base is a potential call.

The ask: distribution into the agent stack, and technical partnership on making audit-before-act a default rather than a decision each developer makes alone.